

CSS Display - Block and Inline Elements

A **block element** is an element that takes up the **full width available**, and has a **line break** before and after it. **Examples**

- <h1>
- <p>
- <div>

An inline element only takes up as much **width as necessary**, and does **not** force **line breaks**.

Examples:

-
- <a>

• Hiding an Element : display:none or visibility:hidden

- space as Hiding an element can be done by setting the **display property** to "**none**" or the **visibility property** to "**hidden**". However, these two methods produce different results:
 - **visibility:hidden** hides an element, but it will still take up the same before. The element will be hidden, but still affect the layout.
visibility:visible displays the element which has been hidden by **visibility:hidden**.
 - **display:none** The element will be hidden, and the page will be displayed as if the element is not there:

Example:

```
h1.hidden {visibility:hidden;}

<!DOCTYPE html>
<html>
<head>
<style type="text/css">
h1.hidden {visibility:hidden;}
</style>
</head>
<body>
<h1>This is a visible heading</h1>
<h1 class="hidden">This is a hidden heading</h1>
<p>Notice that the hidden heading still takes up space.</p>
</body>
</html>
```

- **display:none**

Example:

```
h1.hidden {display:none;}

<!DOCTYPE html>
<html>
<head>
<style type="text/css">
h1.hidden {display:none;}
</style>
</head>
<body>
<h1>This is a visible heading</h1>
```

```
<h1 class="hidden">This is a hidden heading</h1>
  <p>Notice that the hidden heading does not
    take up space.</p>
</body>
</html>
```

Changing How an Element is Displayed

Changing an inline element to a block element, or vice versa, can be useful for making the page look a specific way, and still follow web standards.

The following example displays list items as inline elements:

Example

```
li {display:inline;}
```

The following example displays span elements as block elements:

Example

```
span {display:block;}
```

Positioning

The CSS positioning properties allow you to

- **Position** an element.
- Place an element **behind another**.
- Specify what should happen when an element's **content is too big**.

Elements can be positioned using the **top, bottom, left, and right** properties. However, these properties will not work unless the **position property is set first**. They also work differently depending on the positioning method.

There are **four** different **positioning methods**.

1. Static Positioning

HTML elements are positioned **static by default**. A static positioned element is always **positioned** according to the normal flow of the page.

Static positioned elements are **not affected** by the **top, bottom, left, and right** properties.

2. Fixed Positioning

An element with fixed position is positioned relative to the browser window. It will **not move** even if the **window is scrolled**:

Example [Try](#)

```
p.pos_fixed
{
  position:fixed;
  top:30px;
  right:5px;
}
```

Decide which element to display in front!

Elements can overlap!

Fixed positioned elements are removed from the normal flow. The document and other elements behave like the fixed positioned element does not exist.
Fixed positioned elements can overlap other elements.

3. Relative Positioning

A relative positioned element is positioned relative to its normal position.

Example Try

```
h2.pos_left
{
position:relative;
left:-20px;
}
h2.pos_right
{
position:relative;
left:20px;
}
```

The content of **relatively positioned** elements can be **moved** and **overlap** other elements, but the **reserved space** for the element is still preserved in the normal flow.

Example Try

```
h2.pos_top
{
position:relative;
top:-50px;
}
```

Relatively positioned elements are **often used** as container blocks for absolutely positioned elements.

4. Absolute Positioning

An absolute position element is positioned relative to the first parent element that **has** a **position** other **than static**. If no such element is found, the containing block is <body>:

Example Try

```
h2
{
position:absolute;
left:100px;
top:150px;
}
```

Absolutely positioned elements are removed from the normal flow. The document and other elements behave like the absolutely positioned element does not exist.

Absolutely positioned elements can overlap other elements.

Overlapping Elements

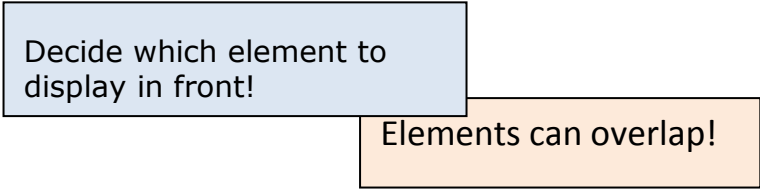
When elements are **positioned outside the normal flow**, they **can overlap** other elements.

The z-index property specifies the stack order of an element (which element should be placed in front of, or behind, the others).

An element can have a positive or negative stack order:

Example [Try](#)

```
img
{
position:absolute;
left:0px;
top:0px;
z-index:-1;
}
```



Decide which element to display in front!

Elements can overlap!

An element with greater stack order is always in front of an element with a lower stack order.

Note: If two positioned elements overlap, without a z-index specified, the element positioned **last** in the HTML code will be **shown on top**.

Overflow property:

How to show overflow in an element using scroll

This example demonstrates how to set the overflow property to create a scroll bar when an element's content is too big to fit in a specified area.

How to set the browser to automatically handle overflow

This example demonstrates how to set the browser to automatically handle overflow.

CSS Float

What is CSS Float?

With CSS float, an element can be pushed to the **left or right**, allowing other **elements to wrap around** it.

Float is very **often used for images**, but it is also useful when working with **layouts**.

How Elements Float

Elements are floated **horizontally**; this means that an element can only be floated **left or right**, **not up or down**.

A floated element will move as far to the left or right as it can. Usually this means all the way to the left or right of the containing element.

The elements **after** the floating element will **flow around** it.

The elements **before** the floating element will **not be affected**.

Example :If an image is floated to the right, the text flows around it, to the left: **Try**

```
img
{
float:right;
}
```

Floating Elements Next to Each Other

If you place several floating elements after each other, they will float next to each other if there is room.

Here we have made an image gallery using the float property:

Example **Try**

```
.thumbnail
{
float:left;
width:110px;
height:90px;
margin:5px;
}
```

Turning off Float - Using Clear

Elements after the floating element will flow around it. To avoid this, use the clear property.

The **clear** property specifies which sides of an element other floating elements are not allowed.

Add a text line into the image gallery, using the clear property:

Example [Try](#)

```
.text_line
{
clear:both;
}
```

More Examples

[An image with border and margins that floats to the right in a paragraph](#)

Let an image float to the right in a paragraph. Add border and margins to the image.

[An image with a caption that floats to the right](#)

Let an image with a caption float to the right.

[Creating a horizontal menu](#)

Use float with a list of hyperlinks to create a horizontal menu.

[Creating a homepage without tables](#)

Use float to create a homepage with a header, footer, left content and main content.

[All the different list item markers in lists](#)

[Fully styled vertical navigation bar](#)

[Styling Forms](#)

Style HTML Elements With Specific Attributes

It is possible to **style** HTML **elements** that have **specific attributes**, not just class and id.

Attribute Selector

The example below styles all elements with a **title attribute**:

Example: [Try](#)

```
[title]
{
color:blue;
}
```

Attribute and Value Selector

This example styles all elements with title="W3Schools":

Example

```
[title=W3Schools]
{
border:5px solid green;
}
```

Attribute and Value Selector - Multiple Values

The example styles all elements with a title attribute that contains a specified value. This works even if the attribute has space separated values:

Example

```
[title~=hello] { color:blue; }
```

CSS **Grouping** and **Nesting** Selectors:

Ex: [CSS Grouping](#) [Nesting Selectors](#)